Documento de Arquitectura Base (DAB)

1. Información General del Proyecto

1.1 Historial de Versiones

Versión	Fecha	Autor(es)	Descripción de los Cambios
1.0	04/07/2026	Gastón Zamora	Creación inicial del Documento de Arquitectura Base. Definición del MVP y stack tecnológico (FastAPI, PostgreSQL, Flutter).

1.2 Participantes Clave

Nombre	Rol en el Proyecto	Responsabilidad Principal
Gastón Zamora	Arquitecto de Software	Diseño del modelo relacional, despliegue cloud y estructura base del proyecto.
Felipe Espinal	Líder Técnico	Desarrollo backend, implementación de API REST y orquestación de servicios.
Martín Crovetto	Gerente General	Definición de reglas de negocio, validación de flujos operativos y aprobación presupuestaria.

[Nombre Profesor/Comisión]	Evaluador Académico (Duoc UC)	Revisión de estándares de ingeniería (ISO 25000, W3C) y validación del portafolio de título.

1.3 Objetivo del Documento

Se detallan las decisiones de diseño técnico necesarias para construir un sistema escalable alojado en la nube que resuelva la problemática actual del negocio: gestionar el flujo completo de deconstrucción vehicular, la trazabilidad unitaria de motores y cajas de cambio mediante códigos QR térmicos, y el control estricto de salidas mediante una barrera digital vinculada a la recaudación en caja.

Este documento servirá como guía única de la verdad (SSOT) para el equipo de desarrollo, asegurando que la construcción de los módulos (Recepción e Ingesta OCR, Bodega WMS, Mesón de Ventas Concurrente, Caja Transaccional con Poka-Yoke, Retiro/Despachos y Módulo Gerencial Zero-Trust) se alinee estrictamente con los atributos de calidad de la norma ISO/IEC 25000 y los estándares de accesibilidad e interoperabilidad de la W3C[cite: 7, 11].

2. Introducción y Contexto

2.1 Alcance de la Arquitectura

El alcance técnico de la arquitectura de Cresal-SaaS define los límites de desarrollo, despliegue y operación del sistema, dividiéndose estrictamente entre los elementos incluidos en el ciclo de vida del proyecto y aquellos explícitamente excluidos para garantizar la viabilidad del desarrollo.

- **Elementos Incluidos (In-Scope):**
    - **Infraestructura Multi-sucursal:** Soporte arquitectónico centralizado en la nube para sincronizar en tiempo real las operaciones de las sucursales de Viña del Mar y Limache.
    - **Módulo de Recepción e Ingesta Inteligente (PDA/Móvil):** Ingesta asistida por Visión Computacional / OCR (Google Cloud Vision API) para extracción automatizada de padrones vehiculares, complementada por scraping vehicular, captura de peritaje fotográfico y generación de códigos SKU con renderizado de QR térmico.
    - **Módulo de Bodega y WMS Industrial (PDA Android):** Cubicaje tridimensional en galpón (Galpón/Pasillo/Pallet/Nivel), gestión de picking, recepción de tareas push y captura obligatoria de motivos en despaletizado.
    - **Módulo de Ventas y Mesón (Windows / Tablet):** Catálogo reactivo mixto (inventariados y misceláneos), prevención de doble venta mediante concurrencia optimista (version_lock) con reservas temporales de 15 minutos

renovadas por Heartbeat, gestión de Ventas en Verde (Backorders) y captura obligatoria de clientes en compras diferidas.
    - **Módulo de Caja Transaccional (Desktop POS):** Panel transaccional multimetodo con aislamiento de concurrencia, transacciones ACID en PostgreSQL, controles anti-error Poka-Yoke (bloqueo de liquidación prematura en abonos, confirmación explícita para pago total) y edición auditada de registros de pago.
    - **Módulo de Logística, Retiro y Envíos:** Control de entregas en mesón y despachos interurbanos vía couriers (Starken, Chilexpress), con arquitectura tolerante a fallos de red en patio mediante caché local y sincronización asíncrona.
    - **Módulo de Barrera Digital y Control de Salida (Patio):** Validación perimetral por doble factor óptico (Match entre QR físico del activo y Token QR de la boleta autorizada) que bloquea la salida sin confirmación financiera.
    - **Módulo Analítico, Auditoría Zero-Trust y Alertas (Fase 2):** Pipeline ETL de migración histórica hacia PostgreSQL, panel de supervisión gerencial con Rol Auditor (solo lectura), KPIs de salidas en tiempo real de motores/cajas, modelo de forecasting predictivo de demanda (Sprint 7) y despacho asíncrono de alertas operativas hacia Gerencia vía bot de Telegram.
    -
- **Elementos Excluidos (Out-of-Scope):**
    - Integración directa mediante hardware con las pasarelas de pago físicas (Transbank/Redelcom). El sistema captura el estado del pago de forma lógica, no procesa la transacción bancaria.
    - Pasarela de venta pública (*E-commerce* o Marketplace) abierta a clientes finales externos en esta fase.
    - Sincronización automatizada con sistemas de contabilidad de terceros o ERPs externos no declarados en el modelo de datos fundacional.

2.2 Objetivos del Negocio

La solución tecnológica propuesta en este documento de arquitectura actúa como el habilitador estratégico para alcanzar los siguientes objetivos operacionales y comerciales de la empresa:

- **Trazabilidad Total del Inventario:** Eliminar la pérdida de activos mediante la vinculación estricta de cada motor a un gemelo digital (SKU/QR) desde su ingreso físico a la desarmaduría.
- **Eficiencia Operativa en Ventas:** Reducir los tiempos de atención en el mesón de ventas al proveer a los ejecutivos un radar de stock dinámico y actualizado al milisegundo a través de dispositivos móviles.
- **Mitigación del Error Humano**: Garantizar mediante software que ningún motor sea etiquetado erróneamente en el ingreso ni despachado de forma ilegal en la barrera de salida, implementando controles digitales forzosos.
- **Descentralización de la Información**: Permitir a la gerencia monitorear la salud del negocio mediante analítica predictiva y facultar a la fuerza de ventas para auditar stock en tiempo real mediante canales conversacionales automatizados.

2.3 Glosario

- **SaaS (Software as a Service):** Modelo de distribución de software donde el soporte y los datos se alojan en servidores de la nube y el cliente accede a través de interfaces web o móviles.
- **MVP (Minimum Viable Product):** Versión de un producto que permite a un equipo recolectar la máxima cantidad de aprendizaje validado con el menor esfuerzo.
- **ACID (Atomicity, Consistency, Isolation, Durability):** Conjunto de propiedades que garantizan que las transacciones en una base de datos se ejecuten de forma segura y fiable.
- **ETL (Extract, Transform, Load):** Proceso de movimiento de datos que extrae información de fuentes origen, la transforma para cumplir con reglas de negocio y la carga en un repositorio analítico.
- **Text-to-SQL:** Subcampo de la inteligencia artificial y el procesamiento de lenguaje natural que traduce instrucciones humanas directamente en consultas SQL ejecutables por un motor de bases de datos.

3. Restricciones y Requerimientos No Funcionales (RNF)

3.1 Restricciones (Constraints)

El diseño y la implementación de la arquitectura de Cresal-SaaS están sujetos a las siguientes limitaciones y condiciones obligatorias:

- **Restricciones de Tiempo:**
  - El ciclo de desarrollo, aseguramiento de calidad (QA) y despliegue del ecosistema transaccional y analítico comprende un marco de 20 semanas (Julio a Noviembre de 2026), culminando con la validación operativa en las sucursales de Viña del Mar y Limache y la entrega formal a la comisión evaluadora.
- **Restricciones Económicas:**
  - El presupuesto total asignado al proyecto es de $2.150.000 CLP[cite: 10]. Este monto se compone de $2.000.000 CLP correspondientes al estipendio pactado para el equipo de desarrollo (5 meses de ejecución a razón de $200.000 CLP mensuales por desarrollador) y $150.000 CLP estimados para el aprovisionamiento de infraestructura Cloud gestionada (Google Cloud Vision API y servicios de Supabase) e insumos de impresión térmica.

3.2 Atributos de Calidad (Basados en ISO/IEC 25000)

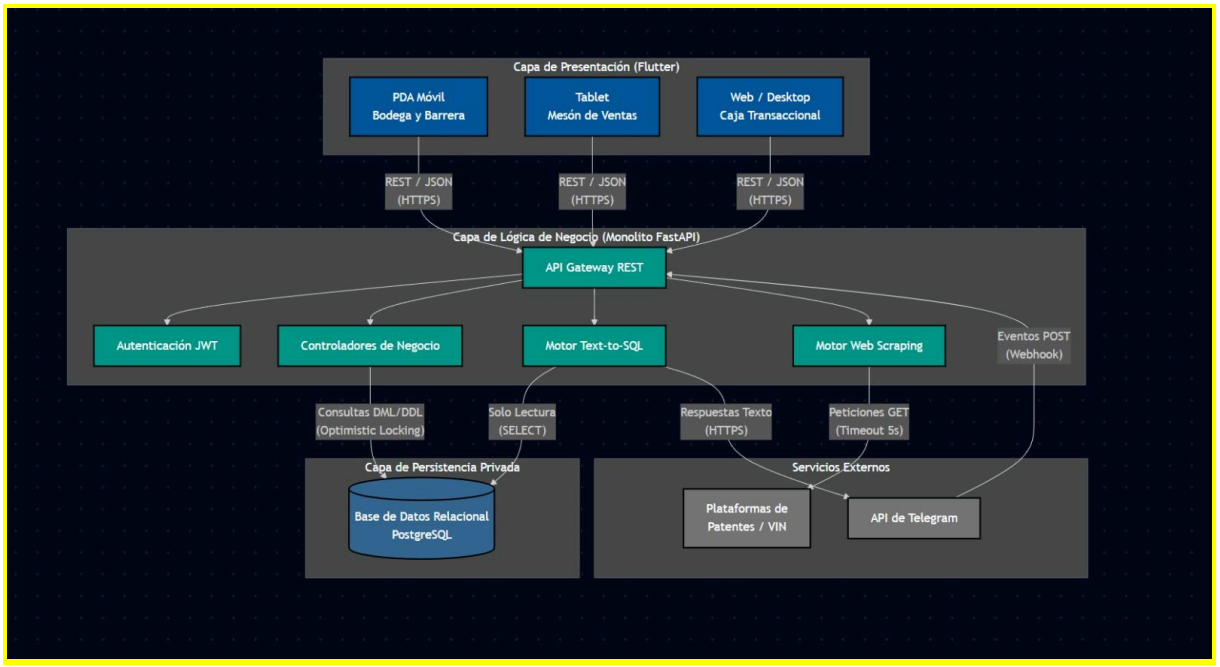
Los siguientes Requerimientos No Funcionales (RNF) definen el comportamiento esperado del sistema y establecen las métricas de calidad que el equipo validará:

- **Rendimiento (Eficiencia de Desempeño):**
  - **RNF-REN-01 (Latencia Transaccional):** Las operaciones de lectura/escritura operacionales deben responder en un tiempo inferior a 500 ms (P95).

    - **RNF-REN-02 (Latencia de Scraping):** El motor de extracción web debe incluir un límite de espera (*Timeout*) estricto de 8.0 segundos. Si la fuente externa no responde, el sistema debe abortar la operación para evitar el agotamiento de recursos.
    - **RNF-REN-03 (Consultas Analíticas):** El panel gerencial y el orquestador predictivo deben procesar el histórico de datos en un tiempo máximo de 2.0 segundos para su renderización inicial en pantalla.
- **Escalabilidad:**
    - **RNF-ESC-01 (Multi-Sucursal):** La arquitectura debe soportar el crecimiento horizontal de entidades físicas, permitiendo agregar nuevas sucursales sin requerir modificaciones en el esquema relacional DDL.
- **Disponibilidad (Fiabilidad):**
    - **RNF-DIS-01 (Uptime Operacional):** El backend transaccional debe garantizar una disponibilidad mínima del 99.5% durante el horario de operación comercial.
    - **RNF-DIS-02 (Tolerancia a Fallos y Baja Señal en Patio):** Las aplicaciones móviles (PDA Android) en Flutter deben retener en almacenamiento local los formularios de recepción y las confirmaciones de retiro/despacho ante pérdidas de señal Wi-Fi o datos móviles en el patio de desarme, aplicando una estrategia de sincronización asíncrona y reintentos automáticos sin bloquear la operación del usuario.
- **Seguridad y Control de Acceso:**
    - **RNF-SEG-01 (Cifrado de Sesiones):** Las sesiones se manejarán exclusivamente mediante Tokens JWT firmados criptográficamente.
    - **RNF-SEG-02 (Aislamiento de Agentes IA):** Las consultas ejecutadas por el módulo *Text-to-SQL* del bot de Telegram utilizarán una conexión a base de datos con permisos estrictos de "Solo Lectura" (`READ ONLY`).
    - **RNF-SEG-03 (Protección en Tránsito):** Toda la comunicación cliente-servidor estará encriptada mediante protocolo HTTPS (TLS 1.2 o superior).

4. Vista Lógica (Arquitectura de Software)

4.1 Diagrama de Componentes y Capas



La arquitectura de Cresal-SaaS se estructura bajo un modelo de capas separadas (N-Tier Architecture), diseñado para maximizar la cohesión y minimizar el acoplamiento entre módulos:

- **Capa de Presentación (Frontend Multiplataforma):** Construida con el SDK de Flutter (Dart). Se encarga exclusivamente de la interfaz de usuario (UI), la gestión del estado local (ej. carritos temporales en la tablet) y la captura de datos físicos a través del hardware del dispositivo (escáner QR de la cámara). Interactúa con el backend de forma estandarizada mediante peticiones HTTP asíncronas.
- **Capa de Lógica de Negocio (Backend API Gateway):** Construida en FastAPI (Python). Actúa como el único punto de entrada (API RESTful) para todas las aplicaciones cliente. Orquesta las reglas de negocio, valida los tokens JWT, ejecuta el motor de web scraping, y gestiona el *webhook* de eventos bidireccionales de Telegram.
- **Capa de Persistencia de Datos:** Soportada por un servidor de base de datos relacional PostgreSQL. Centraliza tanto el almacenamiento transaccional operacional (Fase 1) como el futuro esquema dimensional para inteligencia de negocios (Fase 2).

4.2 Modelo de Datos Relacional

(Instrucción para ti: Insertar aquí la imagen del Diagrama Entidad-Relación - DER que harán en la TSK-1.2)

El sistema emplea un modelo de datos estrictamente relacional para garantizar la integridad referencial de los activos físicos y las operaciones financieras.

- **Gestión de Concurrencia:** Para erradicar colisiones y condiciones de carrera (Race Conditions) al comercializar repuestos únicos no fungibles (ej. dos vendedores intentando asignar el mismo motor simultáneamente), se implementa un patrón de Concurrencia Optimista (Optimistic Locking) mediante version_lock a nivel de base de datos. Este esquema se complementa en la capa de aplicación con reservas efímeras de 15 minutos renovadas periódicamente vía Heartbeat desde las terminales de mesón, asegurando transacciones ACID estrictas en PostgreSQL.
- **Evolución Estructural:** Durante la Fase 1 (MVP), la base de datos mantendrá un esquema fuertemente normalizado (3FN) para operaciones de inserción y lectura rápidas (CRUD) en las entidades principales (`Vehiculo`, `Motor`, `Orden_Pago`, `Vendedor`).

4.3 Patrones de Diseño Arquitectónico

- **Monolito Modular (Backend):** Debido a las restricciones de tiempo y al tamaño del equipo de desarrollo, se descarta inicialmente una arquitectura de microservicios distribuidos. El backend en FastAPI se estructurará como un Monolito Modular, agrupando el código lógicamente por dominios de negocio (Bodega, Ventas, Logística, Analítica) para facilitar una futura extracción a microservicios independientes si la escalabilidad lo exige.
- **Patrón Repositorio (Repository Pattern):** Se implementará en el backend para abstraer la lógica de acceso a datos (consultas SQL) de los controladores de la API. Esto asegura que la lógica de negocio no dependa directamente del driver de la base de datos, mejorando drásticamente la mantenibilidad y facilitando la inyección de dependencias para las pruebas unitarias (*Mocking*).
- **Patrón de Gestión de Estado (Frontend):** En la capa móvil y web, Flutter utilizará un patrón de gestión de estados centralizado (ej. BLoC o Provider) para separar la lógica de presentación de la lógica de negocio local, evitando la reconstrucción innecesaria del árbol de widgets y optimizando el consumo de memoria en dispositivos de gama de entrada.

5. Vista de Integración y APIs

5.1 Interfaces Externas

- **Google Cloud Platform (Vision API - IDP):** Integración de Visión Computacional mediante API REST para el procesamiento inteligente de documentos (IDP). Permite la extracción estructurada de patente, número de chasis (VIN), número de motor, marca, modelo y año a partir de la captura fotográfica del padrón vehicular oficial.
- **Plataformas de Consulta Automotriz (Web Scraping Asistido):** Motor de extracción en Python que consulta portales vehiculares públicos para complementar metadatos técnicos no legibles en padrones deteriorados, operando con un timeout estricto de 8.0 segundos para evitar bloqueos.
- **API de Telegram (Alertas Operativas y Auditoría Forense):** Integración con la API de bots de Telegram para el despacho asíncrono de notificaciones en tiempo real dirigidas a la Gerencia General. Comunica de forma instantánea: liquidación de repuestos inventariados, confirmación de entregas físicas, anulaciones de órdenes (con motivo y responsable) y movimientos de despaletizado en bodega.
- **Hardware Periférico Lógico:** Acceso mediante librerías nativas de Flutter a los componentes de hardware del terminal (cámara para escaneo continuo de códigos QR, sensores de lectura óptica de la PDA industrial e impresoras térmicas de tickets y etiquetas vía Bluetooth/USB).

5.2 Protocolos de Comunicación

- **RESTful API (JSON over HTTPS):** Protocolo principal y exclusivo para la comunicación interna entre las interfaces cliente (Flutter en PDAs, Tablets y Desktop) y el servidor central (FastAPI). Todos los *endpoints* siguen convenciones semánticas de métodos HTTP (GET, POST, PUT, PATCH, DELETE) y envían/reciben cargas útiles estandarizadas en formato JSON, asegurando la interoperabilidad descrita por la W3C.
- **Webhooks Asíncronos:** Para la comunicación con el bot de Telegram, el servidor backend expone un *endpoint* de Webhook. En lugar de consultar constantemente a Telegram por nuevos mensajes (*Long Polling*), el sistema escucha pasivamente de forma asíncrona los eventos HTTP POST que los servidores de Telegram empujan cuando un vendedor envía una consulta, maximizando la eficiencia de los recursos.
- **Resolución Logística Directa:** Dado el alcance del MVP, no se integran *brokers* de mensajería asíncrona complejos (como RabbitMQ o Kafka). La sincronización del inventario y las órdenes entre las ubicaciones de Viña del Mar y Limache se procesa sincrónicamente a través de la API REST centralizada contra la base de datos transaccional.

6. Vista de Despliegue e Infraestructura

6.1 Entorno de Nube (Cloud Environment)

Para el despliegue de Cresal-SaaS, se adopta una arquitectura Cloud-Native basada en servicios gestionados que minimizan la sobrecarga de administración de infraestructura (DevOps) y garantizan alta disponibilidad:

- **Servidor de Aplicaciones (Backend API)**: La API construida en FastAPI se despliega sobre la plataforma PaaS de Render en un contenedor Linux optimizado, gestionando la lógica de negocio, reglas de concurrencia y tareas asíncronas en segundo plano (Background Tasks).
- **Capa de Datos y Persistencia Transaccional:** Se aloja en Supabase Cloud, aprovechando su instancia gestionada de PostgreSQL de alta fidelidad para el almacenamiento transaccional ACID, respaldos automatizados y políticas de seguridad a nivel de fila (Row Level Security - RLS).
- **Almacenamiento Multimedia (Object Storage):** Los registros fotográficos del peritaje vehicular inicial, imágenes de repuestos y evidencias de inspección técnica se almacenan en Supabase Storage (Object Storage compatible con S3), manteniendo en la base de datos relacional únicamente las URLs y metadatos correspondientes.
- **Distribución de Clientes Frontend:**
    - **Terminales Móviles y PDA (Patio y Galpón):** Los aplicativos de Recepción Vehicular y Bodega WMS se empaquetan y compilan como binarios Android nativos (.apk) optimizados para lectura por escáner de hardware y pantallas rugerizadas.
    - **Estaciones Fijas (Mesón, Caja y Gerencia):** Las interfaces de Punto de Venta, Recaudación y Dashboard se compilan como aplicaciones nativas de escritorio para Windows (.exe / .msi), garantizando estabilidad operativa e integración directa con periféricos locales (gavetas de dinero e impresoras térmicas).

6.2 Diagrama de Red y Topología

La topología de red se diseña bajo un principio de **Defensa en Profundidad**:

- **Zona Pública (Internet):** Los dispositivos en las sucursales (Tablets, PDAs, PCs de Caja) y los servidores de Telegram acceden al sistema únicamente a través del protocolo HTTPS apuntando al dominio principal administrado por un *API Gateway* o Balanceador de Carga.
- **Zona Privada (VPC):** El motor de base de datos PostgreSQL se aloja en una subred privada. No posee IP pública, imposibilitando ataques directos desde el exterior. El backend de FastAPI actúa como el único puente autorizado para interactuar con esta zona.

6.3 Estrategia de Integración y Despliegue Continuo (CI/CD)

Para asegurar la calidad del código, mitigar regresiones y automatizar las entregas, se implementará un flujo de *Continuous Integration* y *Continuous Deployment* utilizando **GitHub Actions**.

- **Integración Continua (CI):** Cada vez que un desarrollador realice un *Push* o un *Pull Request* hacia la rama principal (`main` o `develop`), se disparará automáticamente un *pipeline* que ejecutará:
    1. Validación de sintaxis y formateo de código (Linting).
    2. Ejecución de la suite completa de pruebas unitarias (`pytest`) que cubren los módulos críticos (ej. motor de *scraping* y validación financiera).
    3. Generación del reporte de cobertura. Si las pruebas fallan, el código será bloqueado y no podrá fusionarse.
- **Despliegue Continuo (CD):** Una vez que el código aprueba la fase de CI, la integración nativa de la plataforma PaaS detectará los cambios en la rama principal y ejecutará un redespliegue automático del código fuente en el entorno de producción, garantizando que los vendedores y bodegueros siempre cuenten con la última versión estable sin necesidad de interrumpir el servicio (Zero-Downtime Deployment).

7. Matriz de Decisiones Arquitectónicas (ADR)

7.1 Registro de Decisiones Clave

El siguiente registro documenta las justificaciones técnicas fundamentales que guiaron la selección del *stack* tecnológico para el MVP:

- **ADR-01: Selección de PostgreSQL sobre Motores NoSQL (ej. MongoDB)**
    - *Contexto:* El sistema requiere gestionar transacciones financieras (Caja) y trazabilidad física estricta (Bodega y Barrera).
    - *Decisión:* Se elige PostgreSQL.
    - *Justificación:* El modelo de negocio depende de relaciones rígidas (un motor pertenece a un vehículo, una orden tiene un estado financiero). Las garantías ACID de PostgreSQL previenen la corrupción de datos y permiten el uso de bloqueos relacionales (*Optimistic Locking*), lo cual es imposible de garantizar nativamente en entornos NoSQL orientados a documentos.
- **ADR-02: Selección de FastAPI sobre Django/Flask**
    - *Contexto:* Se requiere un backend capaz de manejar flujos de datos asíncronos (web scraping y webhooks de Telegram).
    - *Decisión:* Se elige FastAPI (Python).
    - *Justificación:* FastAPI soporta operaciones asíncronas (`async/await`) de forma nativa desde su núcleo. Además, auto-genera la documentación de la API (Swagger/OpenAPI) basándose en los modelos de Pydantic, lo que acelera drásticamente la integración con el equipo de Frontend (Flutter) y asegura que los contratos de datos se cumplan.

- **ADR-03: Selección de Flutter sobre Desarrollo Nativo (Kotlin/Swift)**
    - *Contexto:* El equipo cuenta con recursos de desarrollo limitados y necesita desplegar aplicaciones tanto para hardware móvil (PDAs y Tablets) como para estaciones de trabajo (Caja/Dashboard).
    - *Decisión:* Se elige Flutter.
    - *Justificación:* Permite mantener un único código base (*Single Codebase*) para compilar binarios nativos tanto en Android como en Web/Desktop. Esto reduce los tiempos de desarrollo a la mitad, garantizando una experiencia de usuario (UI/UX) idéntica en todos los dispositivos de la sucursal.
- **ADR-04: Adopción de Google Cloud Vision API para Ingesta Asistida (IDP)**
    - Contexto: El ingreso manual de padrones en patio generaba demoras operativas y recurrentes errores humanos en la digitación de números de motor y VIN.
    - Decisión: Integrar OCR en la nube a través de Google Cloud Vision API para el procesamiento de documentos vehiculares.
    - Justificación: Provee una tasa de precisión superior al 95% en documentos impresos con tipografía oficial chilena, evitando sobrecargar el procesamiento local de las PDAs y eliminando la necesidad de entrenar y mantener un modelo de visión artificial propio.
- **ADR-05: Selección de Supabase (PostgreSQL Gestionado) para la Persistencia Transaccional**
    - Contexto: Se requería un motor relacional robusto con garantías ACID, escalabilidad, respaldos automáticos y capacidad de almacenamiento de evidencias multimedia sin incurrir en costos de mantenimiento de servidores dedicados.
    - Decisión: Utilizar Supabase Cloud como plataforma central de persistencia.
    - Justificación: Entrega una instancia nativa de PostgreSQL con extensiones avanzadas, soporte directo para conexiones asíncronas desde FastAPI, políticas de acceso RLS y un servicio de Object Storage unificado bajo el mismo ecosistema.